

Лабораторна робота №2

ПОРІВНЯННЯ МЕТОДІВ КЛАСИФІКАЦІЇ ДАНИХ

Мета роботи: використовуючи спеціалізовані бібліотеки та мову програмування Python дослідити різні методи класифікації даних та навчитися їх порівнювати.

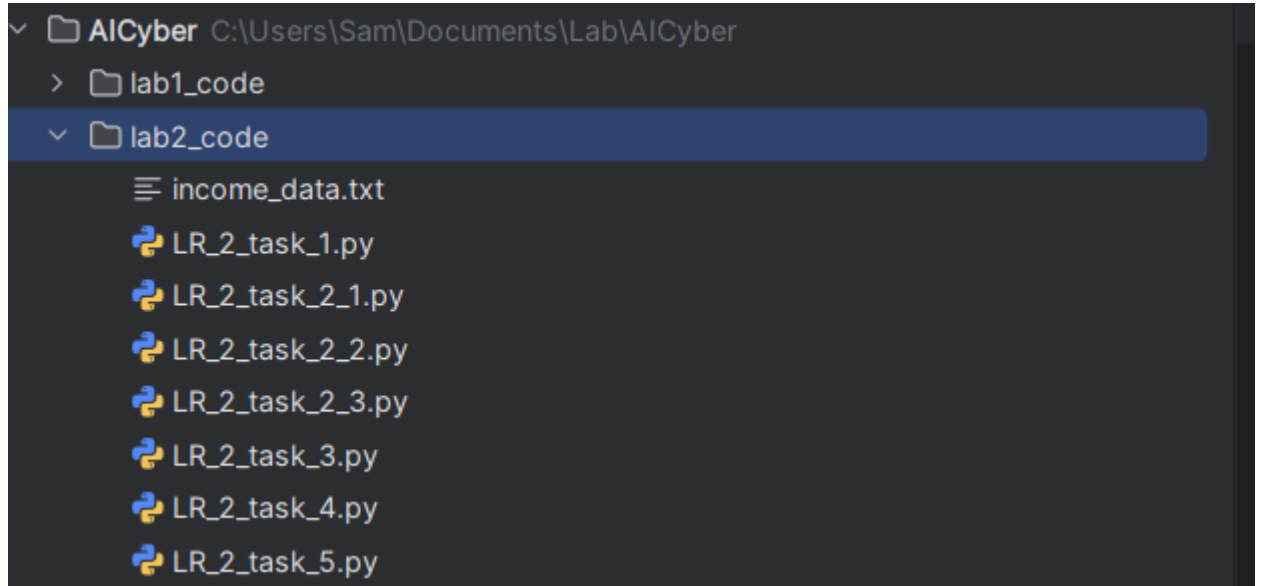


Рисунок – Список всіх завдань на python.

Завдання 2.1 — SVM для доходів

№	Ознака	Значення	Тип
1	age	вік людини	числова
2	workclass	тип зайнятості	категоріальна
3	fnlwgt	ваговий коефіцієнт запису	числова
4	education	рівень освіти	категоріальна
5	education-num	кількість років навчання	числова
6	marital-status	сімейний стан	категоріальна
7	occupation	професія	категоріальна
8	relationship	роль у сім'ї	категоріальна
9	race	раса	категоріальна
10	sex	стать	категоріальна
11	capital-gain	приріст капіталу	числова
12	capital-loss	втрата капіталу	числова
13	hours-per-week	годин роботи на тиждень	числова
14	native-country	країна походження	категоріальна

Таблица 1 - 14 ознак набору Census Income

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.25.125.12.000 –Лр.2					
Змн.	Арк.	№ докум.	Підпис	Дата						
Розроб.		Лук'яненко В. Я.			Звіт з лабораторної роботи №2			Літ.	Арк.	Акрушів
Перевір.		Маєвський О. В.							1	
Реценз.								ФІКТ, гр. КБ-23-1		
Н. Контр.										
Зав.каф.		Єфіменко А.А.								

ЛІСТИНГ КОД :

```
import numpy as np
from sklearn import preprocessing
from sklearn.svm import LinearSVC
from sklearn.multiclass import OneVsOneClassifier
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, classification_report

input_file = "income_data.txt"

X = []
count_class1 = 0
count_class2 = 0
max_datapoints = 25000

with open(input_file, "r") as f:
    for line in f.readlines():
        if count_class1 >= max_datapoints and count_class2 >= max_datapoints:
            break

        if "?" in line:
            continue

        data = line.strip().split(", ")

        if data[-1] == "<=50K" and count_class1 < max_datapoints:
            X.append(data)
            count_class1 += 1

        elif data[-1] == ">50K" and count_class2 < max_datapoints:
            X.append(data)
            count_class2 += 1

X = np.array(X)

label_encoder = []
X_encoded = np.empty(X.shape)

for i, item in enumerate(X[0]):
    if item.replace("-", "").isdigit():
        X_encoded[:, i] = X[:, i]
    else:
        le = preprocessing.LabelEncoder()
        X_encoded[:, i] = le.fit_transform(X[:, i])
        label_encoder.append(le)

X_data = X_encoded[:, :-1].astype(int)
y = X_encoded[:, -1].astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X_data, y, test_size=0.2, random_state=5
)

classifier = OneVsOneClassifier(LinearSVC(random_state=0, max_iter=10000))
classifier.fit(X_train, y_train)
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.125.12.000 –Лр.2	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		2

```

y_test_pred = classifier.predict(X_test)

accuracy = accuracy_score(y_test, y_test_pred)
precision = precision_score(y_test, y_test_pred, average="weighted")
recall = recall_score(y_test, y_test_pred, average="weighted")
f1 = f1_score(y_test, y_test_pred, average="weighted")

print("Accuracy:", round(accuracy * 100, 2), "%")
print("Precision:", round(precision * 100, 2), "%")
print("Recall:", round(recall * 100, 2), "%")
print("F1 score:", round(f1 * 100, 2), "%")

print("\nClassification report:")
print(classification_report(y_test, y_test_pred))

cv_f1 = cross_val_score(classifier, X_data, y, scoring="f1_weighted", cv=3)
print("Cross-validation F1:", round(100 * cv_f1.mean(), 2), "%")

input_data = [
    "37", "Private", "215646", "HS-grad", "9", "Never-married",
    "Handlers-cleaners", "Not-in-family", "White", "Male",
    "0", "0", "40", "United-States"
]

input_data_encoded = []
encoder_index = 0

for item in input_data:
    if item.replace("-", "").isdigit():
        input_data_encoded.append(int(item))
    else:
        encoded_value = label_encoder[encoder_index].transform([item])[0]
        input_data_encoded.append(encoded_value)
        encoder_index += 1

input_data_encoded = np.array(input_data_encoded).reshape(1, -1)

predicted_class = classifier.predict(input_data_encoded)
result = label_encoder[-1].inverse_transform(predicted_class)

print("\nTest point prediction:", result[0])

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.125.12.000 –Лр.2	Арк.
						3
Змн.	Арк.	№ докум.	Підпис	Дата		

```
C:\Users\Sam\AppData\Local\Programs\Python\Python313\python.exe C:\Users\Sam\Documents\Lab\AICyber\lab2_code\LR_2_task_1.py
Accuracy: 79.56 %
Precision: 79.26 %
Recall: 79.56 %
F1 score: 75.75 %

Classification report:
      precision    recall  f1-score   support

     0       0.80      0.97      0.88       4492
     1       0.78      0.28      0.41       1541

   accuracy          0.80       6033
  macro avg       0.79      0.63      0.64       6033
weighted avg       0.79      0.80      0.76       6033

Cross-validation F1: 76.01 %

Test point prediction: <=50K
```

Рисунок – Результат виконання програми.

У завданні було створено SVM-класифікатор з лінійним ядром для прогнозування рівня доходу людини за 14 ознаками. Категоріальні ознаки були закодовані за допомогою LabelEncoder, а числові ознаки залишені без змін. Після навчання модель прогнозує, чи належить людина до класу доходу $\leq 50K$ або $> 50K$.

Завдання 2.2 — SVM з нелінійними ядрами

Перевірка поліноміальне ядро.

Лістинг код :

```
from LR_2_task_1 import X_data, y

from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

# Беремо менше даних, щоб програма не зависала
X_small = X_data[:5000]
y_small = y[:5000]

X_train, X_test, y_train, y_test = train_test_split(
    X_small, y_small, test_size=0.2, random_state=5
)
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.125.12.000 –Лр.2	Арк.
						4
Змн.	Арк.	№ докум.	Підпис	Дата		

```

classifier = make_pipeline(
    StandardScaler(),
    SVC(kernel="poly", degree=3, C=1.0, gamma="scale", max_iter=50000)
)

classifier.fit(X_train, y_train)

y_pred = classifier.predict(X_test)

print("Polynomial SVM")
print("Accuracy:", round(accuracy_score(y_test, y_pred) * 100, 2), "%")
print("Precision:", round(precision_score(y_test, y_pred, average="weighted") *
100, 2), "%")
print("Recall:", round(recall_score(y_test, y_pred, average="weighted") * 100, 2),
"%")
print("F1:", round(f1_score(y_test, y_pred, average="weighted") * 100, 2), "%")

```

```

test point prediction: 4/600
Polynomial SVM
Accuracy: 81.1 %
Precision: 81.43 %
Recall: 81.1 %
F1: 78.45 %

```

Рисунок – Результат виконання команди.

Перевірка гаусове ядро.

Лістинг код :

```

from LR_2_task_1 import X_data, y
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score

X_train, X_test, y_train, y_test = train_test_split(
    X_data, y, test_size=0.2, random_state=5
)

classifier = SVC(kernel="rbf")
classifier.fit(X_train, y_train)

y_pred = classifier.predict(X_test)

print("RBF SVM")
print("Accuracy:", round(accuracy_score(y_test, y_pred) * 100, 2), "%")
print("Precision:", round(precision_score(y_test, y_pred, average="weighted") *
100, 2), "%")
print("Recall:", round(recall_score(y_test, y_pred, average="weighted") * 100, 2),

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.125.12.000 –Лр.2	Арк.
						5
Змн.	Арк.	№ докум.	Підпис	Дата		

```
"%")
print("F1:", round(f1_score(y_test, y_pred, average="weighted") * 100, 2), "%")
```

```
RBF SVM
Accuracy: 78.19 %
Precision: 82.82 %
Recall: 78.19 %
F1: 71.51 %
```

Рисунок – Результат виконання програми.

Перевірка сигмоїдальне ядро.

Лістинг код :

```
from LR_2_task_1 import X_data, y
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score

X_train, X_test, y_train, y_test = train_test_split(
    X_data, y, test_size=0.2, random_state=5
)

classifier = SVC(kernel="sigmoid")
classifier.fit(X_train, y_train)

y_pred = classifier.predict(X_test)

print("Sigmoid SVM")
print("Accuracy:", round(accuracy_score(y_test, y_pred) * 100, 2), "%")
print("Precision:", round(precision_score(y_test, y_pred, average="weighted") *
100, 2), "%")
print("Recall:", round(recall_score(y_test, y_pred, average="weighted") * 100, 2),
"%")
print("F1:", round(f1_score(y_test, y_pred, average="weighted") * 100, 2), "%")
```

```
Sigmoid SVM
Accuracy: 60.47 %
Precision: 60.64 %
Recall: 60.47 %
F1: 60.55 %
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.125.12.000 –Лр.2	Арк.
						6
Змн.	Арк.	№ докум.	Підпис	Дата		

Рисунок – Результат виконання програми.

Було досліджено три нелінійні SVM-класифікатори: з поліноміальним, гаусовим та сигмоїдальним ядром. Найкращим вважається той класифікатор, який має найбільші значення Accuracy, Precision, Recall та F1.

Завдання 2.3 — Iris dataset

Лістинг код :

```
from pandas import read_csv
from pandas.plotting import scatter_matrix
from matplotlib import pyplot

from sklearn.model_selection import train_test_split, cross_val_score,
StratifiedKfold
from sklearn.metrics import classification_report, confusion_matrix,
accuracy_score
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC

import numpy as np

url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/iris.csv"
names = ["sepal-length", "sepal-width", "petal-length", "petal-width", "class"]
dataset = read_csv(url, names=names)

print("Shape:")
print(dataset.shape)

print("\nFirst 20 rows:")
print(dataset.head(20))

print("\nDescription:")
print(dataset.describe())

print("\nClass distribution:")
print(dataset.groupby("class").size())

dataset.plot(kind="box", subplots=True, layout=(2, 2), sharex=False, sharey=False)
pyplot.suptitle("Boxplot of Iris attributes")
pyplot.savefig("iris_boxplot.png")
pyplot.show()

dataset.hist()
pyplot.suptitle("Histogram of Iris attributes")
pyplot.savefig("iris_histogram.png")
pyplot.show()

scatter_matrix(dataset)
pyplot.suptitle("Scatter matrix of Iris dataset")
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.125.12.000 –Лр.2	Арк.
						7
Змн.	Арк.	№ докум.	Підпис	Дата		

```

pyplot.savefig("iris_scatter_matrix.png")
pyplot.show()

array = dataset.values
X = array[:, 0:4]
Y = array[:, 4]

X_train, X_validation, Y_train, Y_validation = train_test_split(
    X, Y, test_size=0.20, random_state=1
)

models = []
models.append(("LR", LogisticRegression(solver="liblinear", multi_class="ovr")))
models.append(("LDA", LinearDiscriminantAnalysis()))
models.append(("KNN", KNeighborsClassifier()))
models.append(("CART", DecisionTreeClassifier()))
models.append(("NB", GaussianNB()))
models.append(("SVM", SVC(gamma="auto")))

results = []
names = []

print("\nModel comparison:")
for name, model in models:
    kfold = StratifiedKFold(n_splits=10, random_state=1, shuffle=True)
    cv_results = cross_val_score(model, X_train, Y_train, cv=kfold,
    scoring="accuracy")
    results.append(cv_results)
    names.append(name)
    print("%s: %f (%f)" % (name, cv_results.mean(), cv_results.std()))

pyplot.boxplot(results, labels=names)
pyplot.title("Algorithm Comparison")
pyplot.savefig("iris_algorithm_comparison.png")
pyplot.show()

model = SVC(gamma="auto")
model.fit(X_train, Y_train)

predictions = model.predict(X_validation)

print("\nAccuracy on validation:")
print(accuracy_score(Y_validation, predictions))

print("\nConfusion matrix:")
print(confusion_matrix(Y_validation, predictions))

print("\nClassification report:")
print(classification_report(Y_validation, predictions))

X_new = np.array([[5, 2.9, 1, 0.2]])
prediction = model.predict(X_new)

print("\nNew flower:")
print("X_new shape:", X_new.shape)
print("Prediction:", prediction[0])

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.125.12.000 –Лр.2	Арк.
						8
Змн.	Арк.	№ докум.	Підпис	Дата		


```
Shape:
(150, 5)

First 20 rows:
  sepal-length  sepal-width  petal-length  petal-width  class
0           5.1           3.5           1.4           0.2  Iris-setosa
1           4.9           3.0           1.4           0.2  Iris-setosa
2           4.7           3.2           1.3           0.2  Iris-setosa
3           4.6           3.1           1.5           0.2  Iris-setosa
4           5.0           3.6           1.4           0.2  Iris-setosa
5           5.4           3.9           1.7           0.4  Iris-setosa
6           4.6           3.4           1.4           0.3  Iris-setosa
7           5.0           3.4           1.5           0.2  Iris-setosa
8           4.4           2.9           1.4           0.2  Iris-setosa
9           4.9           3.1           1.5           0.1  Iris-setosa
10          5.4           3.7           1.5           0.2  Iris-setosa
11          4.8           3.4           1.6           0.2  Iris-setosa
12          4.8           3.0           1.4           0.1  Iris-setosa
13          4.3           3.0           1.1           0.1  Iris-setosa
14          5.8           4.0           1.2           0.2  Iris-setosa
15          5.7           4.4           1.5           0.4  Iris-setosa
16          5.4           3.9           1.3           0.4  Iris-setosa
17          5.1           3.5           1.4           0.3  Iris-setosa
18          5.7           3.8           1.7           0.3  Iris-setosa
19          5.1           3.8           1.5           0.3  Iris-setosa

Description:
  sepal-length  sepal-width  petal-length  petal-width
count  150.000000  150.000000  150.000000  150.000000
mean    5.843333    3.054000    3.758667    1.198667
std     0.828066    0.433594    1.764420    0.763161
min     4.300000    2.000000    1.000000    0.100000
25%     5.100000    2.800000    1.600000    0.300000
50%     5.800000    3.000000    4.350000    1.300000
75%     6.400000    3.300000    5.100000    1.800000
max     7.900000    4.400000    6.900000    2.500000

Class distribution:
class
Iris-setosa      50
Iris-versicolor  50
Iris-virginica   50
dtype: int64
```

Результат – логівання процесу виконання завдання.

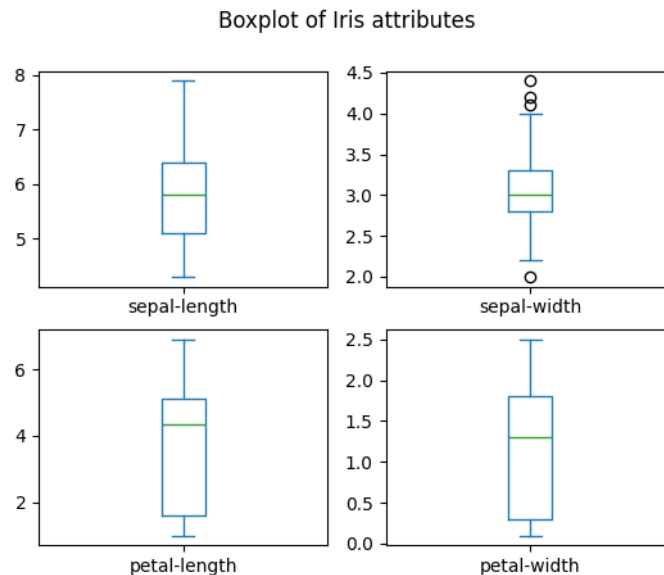


Рисунок – Результат виконання даної програми.

Було досліджено набір Iris, який містить 150 зразків ірисів трьох класів: setosa, versicolor та virginica. Кожен зразок описується чотирма ознаками: довжина чашолистка, ширина чашолистка, довжина пелюстки та ширина пелюстки. Було порівняно шість алгоритмів класифікації: Logistic Regression, LDA, KNN, CART, Naive Bayes та SVM. За результатами крос-валідації найкращим можна вважати алгоритм з найбільшим середнім значенням accuracy. **Iris-setosa**.

Завдання 2.4 — порівняння алгоритмів для income_data.txt

Лістинг код :

```
import numpy as np
from sklearn import preprocessing
from sklearn.model_selection import train_test_split, cross_val_score,
StratifiedKfold
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from matplotlib import pyplot

input_file = "income_data.txt"

X = []
count_class1 = 0
count_class2 = 0
max_datapoints = 5000
```

```

with open(input_file, "r") as f:
    for line in f.readlines():
        if count_class1 >= max_datapoints and count_class2 >= max_datapoints:
            break

        if "?" in line:
            continue

        data = line.strip().split(", ")

        if data[-1] == "<=50K" and count_class1 < max_datapoints:
            X.append(data)
            count_class1 += 1

        elif data[-1] == ">50K" and count_class2 < max_datapoints:
            X.append(data)
            count_class2 += 1

X = np.array(X)

label_encoder = []
X_encoded = np.empty(X.shape)

for i, item in enumerate(X[0]):
    if item.replace("-", "").isdigit():
        X_encoded[:, i] = X[:, i]
    else:
        le = preprocessing.LabelEncoder()
        X_encoded[:, i] = le.fit_transform(X[:, i])
        label_encoder.append(le)

X_data = X_encoded[:, :-1].astype(int)
y = X_encoded[:, -1].astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X_data, y, test_size=0.2, random_state=1
)

models = []
models.append(("LR", LogisticRegression(solver="liblinear", max_iter=1000)))
models.append(("LDA", LinearDiscriminantAnalysis()))
models.append(("KNN", KNeighborsClassifier()))
models.append(("CART", DecisionTreeClassifier()))
models.append(("NB", GaussianNB()))
models.append(("SVM", SVC(gamma="auto")))

results = []
names = []

print("Comparison of classifiers for income_data.txt:\n")

for name, model in models:
    kfold = StratifiedKFold(n_splits=10, random_state=1, shuffle=True)
    cv_results = cross_val_score(model, X_train, y_train, cv=kfold,
    scoring="accuracy")

    model.fit(X_train, y_train)

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.125.12.000 –Лр.2	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		11

```
y_pred = model.predict(X_test)

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, average="weighted")
rec = recall_score(y_test, y_pred, average="weighted")
f1 = f1_score(y_test, y_pred, average="weighted")

results.append(cv_results)
names.append(name)

print(name)
print("Cross-validation accuracy:", round(cv_results.mean() * 100, 2), "%")
print("Accuracy:", round(acc * 100, 2), "%")
print("Precision:", round(prec * 100, 2), "%")
print("Recall:", round(rec * 100, 2), "%")
print("F1:", round(f1 * 100, 2), "%")
print("-" * 40)

pyplot.boxplot(results, tick_labels=names)
pyplot.title("Income Dataset Algorithm Comparison")
pyplot.savefig("income_algorithm_comparison.png")
pyplot.show()
```

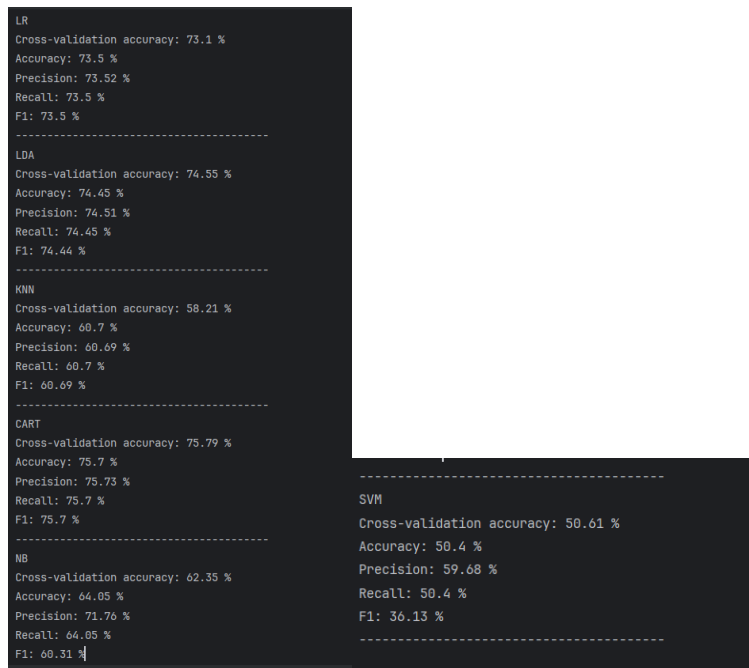


Рисунок – Результат порівняння алгоритмів

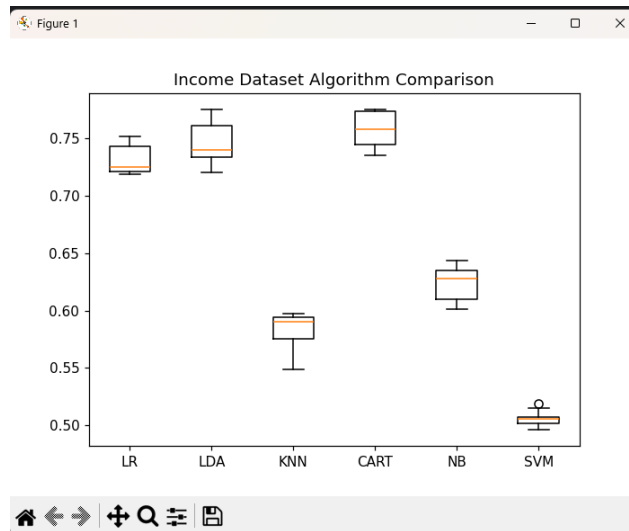


Рисунок – Візуальне порівняння даних.

Було виконано порівняння шести алгоритмів класифікації для набору income_data.txt: Logistic Regression, LDA, KNN, CART, Naive Bayes та SVM. Для кожного алгоритму розраховано ассигасу, precision, recall та F1-score. Найкращим алгоритмом потрібно вважати той, який має найбільше значення F1-score та ассигасу, тому що він не лише правильно класифікує більшість прикладів, а й краще враховує баланс між точністю та повнотою. Алгоритм **CART**

Завдання 2.5 — RidgeClassifier

Лістинг код :

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.linear_model import RidgeClassifier
from sklearn.model_selection import train_test_split
from sklearn import metrics
from sklearn.metrics import confusion_matrix
from io import BytesIO

iris = load_iris()

X = iris.data
y = iris.target

Xtrain, Xtest, ytrain, ytest = train_test_split(
    X, y, test_size=0.3, random_state=0
)

clf = RidgeClassifier(tol=1e-2, solver="sag")
```

```

clf.fit(Xtrain, ytrain)

ypred = clf.predict(Xtest)

print("Accuracy:", np.round(metrics.accuracy_score(ytest, ypred), 4))
print("Precision:", np.round(metrics.precision_score(ytest, ypred,
average="weighted"), 4))
print("Recall:", np.round(metrics.recall_score(ytest, ypred, average="weighted"),
4))
print("F1 Score:", np.round(metrics.f1_score(ytest, ypred, average="weighted"),
4))
print("Cohen Kappa Score:", np.round(metrics.cohen_kappa_score(ytest, ypred), 4))
print("Matthews Corrcoef:", np.round(metrics.matthews_corrcoef(ytest, ypred), 4))

print("\nClassification Report:\n")
print(metrics.classification_report(ytest, ypred, target_names=iris.target_names))

mat = confusion_matrix(ytest, ypred)

sns.set()
sns.heatmap(mat.T, square=True, annot=True, fmt="d", cbar=False)

plt.xlabel("true label")
plt.ylabel("predicted label")
plt.title("Confusion Matrix for Ridge Classifier")
plt.savefig("Confusion.jpg")
plt.show()

f = BytesIO()
plt.savefig(f, format="svg")

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.125.12.000 –Лр.2	Арк.
						14
Змн.	Арк.	№ докум.	Підпис	Дата		

```

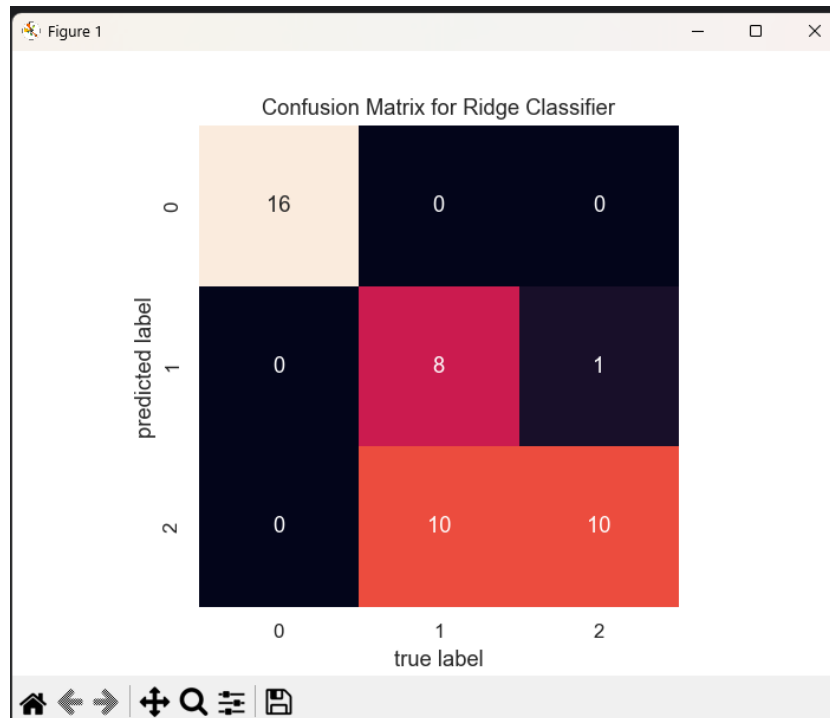
Accuracy: 0.7556
Precision: 0.8333
Recall: 0.7556
F1 Score: 0.7503
Cohen Kappa Score: 0.6431
Matthews Corrccoef: 0.6831

```

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	16
versicolor	0.89	0.44	0.59	18
virginica	0.50	0.91	0.65	11
accuracy			0.76	45
macro avg	0.80	0.78	0.75	45
weighted avg	0.83	0.76	0.75	45

Рисунок – реалізовано класифікацію набору Iris за допомогою лінійного класифікатора Ridge



Було реалізовано класифікацію набору Iris за допомогою лінійного класифікатора Ridge. Модель була навчена на 70% даних, а перевірена на 30% тестової вибірки. Для оцінки якості були використані accuracy, precision, recall, F1-score, Cohen Kappa Score та Matthews Corrocoef. Матриця плутанини показує, скільки об'єктів кожного класу було класифіковано правильно і де модель допустила помилки.

Посилання на **GitHub**